PA 446 Midterm

Pablo R Valenzuela

2026-10-18

Part I: Repository and Workflow

Part II: Web Scraping

Question 1: Scrape a Table

```
# Loading packages used for web scraping and data cleaning per the lectures
library(rvest)
```

```
## Warning: package 'rvest' was built under R version 4.5.3
```

```
library(dplyr)
```

```
## Warning: package 'dplyr' was built under R version 4.5.3
```

```
##
## Attaching package: 'dplyr'
```

```
## The following objects are masked from 'package:stats':
##
##     filter, lag
```

```
## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union
```

```
library(janitor)
```

```
## Warning: package 'janitor' was built under R version 4.5.3
```

```
##
## Attaching package: 'janitor'
```

```
## The following objects are masked from 'package:stats':
##
##     chisq.test, fisher.test
```

```
library(stringr)
```

```
## Warning: package 'stringr' was built under R version 4.5.3
```

```
# Reading the Wikipedia page containing Chicago community areas

url <- "https://en.wikipedia.org/wiki/Community_areas_in_Chicago"

community_page <- read_html(url)

# Extract all tables from the page and let's inspect them

tables <- community_page %>%
  html_elements("table") %>%
  html_table()

# Display the number of tables found

length(tables)
```

```
## [1] 5
```

```
# we have 5 tables displaying. The likely table we need is #2 with 78 rows
# and 13 columns
```

```
# Examine the table that contains Chicago community areas

head(tables[[2]])
```

```
## # A tibble: 6 × 13
##     No. Name           `Total Population` White  `Hispanic/Latino` Black  Asian
##   <int> <chr>          <chr>              <chr>  <chr>             <chr>  <chr>
## 1     1 Rogers Park    54,173             24,150 12,245            11,729 3,389
## 2     2 West Ridge     78,373             30,859 16,129            9,180  17,104
## 3     3 Uptown         56,344             29,284 7,971             10,713 5,831
## 4     4 Lincoln Square 41,651             25,958 7,405             1,732  3,814
## 5     5 North Center   36,014             26,413 4,267             846    2,239
## 6     6 Lake View      102,827            74,953 10,539            4,603  7,480
## # ℹ 6 more variables: `Other/Multiple Races` <chr>, `% White` <dbl>,
## #   `% Hispanic/Latino` <dbl>, `% Black` <dbl>, `% Asian` <dbl>,
## #   `% Other/Multiple Races` <dbl>
```

```
# I'm now storing the community areas table and cleaning the column names

community_areas <- tables[[2]] %>%
  clean_names()

# Displaying column names

names(community_areas)
```

```
##  [1] "no"                          "name"
##  [3] "total_population"            "white"
##  [5] "hispanic_latino"             "black"
##  [7] "asian"                       "other_multiple_races"
##  [9] "percent_white"               "percent_hispanic_latino"
## [11] "percent_black"               "percent_asian"
## [13] "percent_other_multiple_races"
```

```
# Displaing dimensions

dim(community_areas)
```

```
## [1] 78 13
```

```
# All column names match the Wikipedia site :)
```

```
# Now I display the last rows of the table to locate the citywide total row. Checked
# for the tail command to inspect what is the name of the last row. It's named
# Chicago

tail(community_areas)
```

```
## # A tibble: 6 × 13
##      no name               total_population white   hispanic_latino black  asian
##   <int> <chr>              <chr>            <chr>   <chr>           <chr>  <chr>
## 1    73 Washington Heights 24,851           350     915             23,100 70
## 2    74 Mount Greenwood    19,059           15,698  1,782           668    134
## 3    75 Morgan Park        20,873           5,982   1,187           12,760 265
## 4    76 O'Hare             14,394           8,915   1,648           881    2,066
## 5    77 Edgewater          56,563           29,273  9,267           6,965  7,660
## 6    NA Chicago            2,711,226        870,127 805,318         743,7… 196,…
## # ℹ 6 more variables: other_multiple_races <chr>, percent_white <dbl>,
## #   percent_hispanic_latino <dbl>, percent_black <dbl>, percent_asian <dbl>,
## #   percent_other_multiple_races <dbl>
```

```
# I now remove the citywide Chicago total row

community_areas <- community_areas %>%
  filter(name != "Chicago")

# Now I must convert total population from character to numeric value

community_areas <- community_areas %>%
  mutate(
    total_population = as.numeric(
      str_remove_all(total_population, ",")
    )
  )

# Checking there are now 77 community areas

dim(community_areas)
```

```
## [1] 77 13
```

```
# Now I want to show the row for West Town

community_areas %>%
  filter(name == "West Town")
```

```
## # A tibble: 1 × 13
##      no name      total_population white  hispanic_latino black asian
##   <int> <chr>                 <dbl> <chr>  <chr>           <chr> <chr>
## 1    24 West Town             88164 54,160 17,452          5,355 5,812
## # ℹ 6 more variables: other_multiple_races <chr>, percent_white <dbl>,
## #   percent_hispanic_latino <dbl>, percent_black <dbl>, percent_asian <dbl>,
## #   percent_other_multiple_races <dbl>
```

```
# Display the 5 community areas with the largest populations

community_areas %>%
  arrange(desc(total_population)) %>%
  head(5)
```

```
## # A tibble: 5 × 13
##      no name            total_population white  hispanic_latino black  asian
##   <int> <chr>                      <dbl> <chr>  <chr>           <chr>  <chr>
## 1     8 Near North Side           107331 69,049 8,884           8,520  15,939
## 2     6 Lake View                 102827 74,953 10,539          4,603  7,480
## 3    25 Austin                     97452 5,452  21,212          67,261 830
## 4    24 West Town                  88164 54,160 17,452          5,355  5,812
## 5     2 West Ridge                 78373 30,859 16,129          9,180  17,104
## # ℹ 6 more variables: other_multiple_races <chr>, percent_white <dbl>,
## #   percent_hispanic_latino <dbl>, percent_black <dbl>, percent_asian <dbl>,
## #   percent_other_multiple_races <dbl>
```

Question 2: Scrape with CSS selectors

```
# Reading the Books to Scrape site

books_url <- "https://books.toscrape.com/"

books_page <- read_html(books_url)
```

```
# Extracting  all book containers from the webpage

books <- books_page %>%
  html_elements("article.product_pod")

length(books)
```

```
## [1] 20
```

```
# Now we have the 20 books, we must extract the title of each book

titles <- books %>%
  html_element("h3 a") %>%
  html_attr("title")

# Let's take a look at the first few titles or the entire list to be sure

head(titles)
```

```
## [1] "A Light in the Attic"
## [2] "Tipping the Velvet"
## [3] "Soumission"
## [4] "Sharp Objects"
## [5] "Sapiens: A Brief History of Humankind"
## [6] "The Requiem Red"
```

```
titles
```

```
##  [1] "A Light in the Attic"
##  [2] "Tipping the Velvet"
##  [3] "Soumission"
##  [4] "Sharp Objects"
##  [5] "Sapiens: A Brief History of Humankind"
##  [6] "The Requiem Red"
##  [7] "The Dirty Little Secrets of Getting Your Dream Job"
##  [8] "The Coming Woman: A Novel Based on the Life of the Infamous Feminist, Victori
a Woodhull"
##  [9] "The Boys in the Boat: Nine Americans and Their Epic Quest for Gold at the 193
6 Berlin Olympics"
## [10] "The Black Maria"
## [11] "Starving Hearts (Triangular Trade Trilogy, #1)"
## [12] "Shakespeare's Sonnets"
## [13] "Set Me Free"
## [14] "Scott Pilgrim's Precious Little Life (Scott Pilgrim #1)"
## [15] "Rip it Up and Start Again"
## [16] "Our Band Could Be Your Life: Scenes from the American Indie Underground, 198
1-1991"
## [17] "Olio"
## [18] "Mesaerion: The Best Science Fiction Stories 1800-1849"
## [19] "Libertarianism for Beginners"
## [20] "It's Only the Himalayas"
```

```
# Extracting the price of each book

prices <- books %>%
  html_element(".price_color") %>%
  html_text2()

# Display first few prices. Matches the prices in the website

head(prices)
```

```
## [1] "£51.77" "£53.74" "£50.10" "£47.82" "£54.23" "£22.65"
```

```
# Extracting availability information

availability <- books %>%
  html_element(".instock") %>%
  html_text2()

# Display first few availability values. It matches the website
head(availability)
```

```
## [1] "In stock" "In stock" "In stock" "In stock" "In stock" "In stock"
```

```
# Combining the scraped information into a tibble

books_tbl <- tibble(
  title = titles,
  price = prices,
  availability = availability
)

# Display the number of rows

nrow(books_tbl)
```

```
## [1] 20
```

```
# Display the first rows

head(books_tbl)
```

```
## # A tibble: 6 × 3
##   title                                price  availability
##   <chr>                                <chr>  <chr>
## 1 A Light in the Attic                 £51.77 In stock
## 2 Tipping the Velvet                   £53.74 In stock
## 3 Soumission                           £50.10 In stock
## 4 Sharp Objects                        £47.82 In stock
## 5 Sapiens: A Brief History of Humankind £54.23 In stock
## 6 The Requiem Red                      £22.65 In stock
```

```
# Now I convert price from text (character data) to numeric (numbers)

books_tbl <- books_tbl %>%
  mutate(
    price = as.numeric(str_remove(price, "£"))
  )

# Verifying the change to show numbers only without the currency sign

head(books_tbl)
```

```
## # A tibble: 6 × 3
##   title                                price availability
##   <chr>                                <dbl> <chr>
## 1 A Light in the Attic                  51.8 In stock
## 2 Tipping the Velvet                    53.7 In stock
## 3 Soumission                            50.1 In stock
## 4 Sharp Objects                         47.8 In stock
## 5 Sapiens: A Brief History of Humankind  54.2 In stock
## 6 The Requiem Red                       22.6 In stock
```

```
# Save the table to a CSV file as required
write.csv(
  books_tbl,
  "books.csv",
  row.names = FALSE
)
```

Bonus Part: Star Ratings

```
#  BONUS QUESTION: Extract star ratings. The star counts from the website match.

ratings <- books %>%
  html_element(".star-rating") %>%
  html_attr("class")

# Display first few ratings

head(ratings)
```

```
## [1] "star-rating Three" "star-rating One"   "star-rating One"
## [4] "star-rating Four"  "star-rating Five"  "star-rating One"
```

```
# Convert text ratings into numeric ratings

ratings <- ratings %>%
  str_remove("star-rating ") %>%
  recode(
    "One" = 1,
    "Two" = 2,
    "Three" = 3,
    "Four" = 4,
    "Five" = 5
  ) %>%
  as.numeric()

# Check the first few ratings to make sure we got numbers instead of spelled out
# numbers

head(ratings)
```

```
## [1] 3 1 1 4 5 1
```

```
# Now that it worked finally, I need to add numeric star rating to the books table

books_tbl <- books_tbl %>%
  mutate(star_rating = ratings)

# Display first rows
head(books_tbl)
```

```
## # A tibble: 6 × 4
##   title                                price availability star_rating
##   <chr>                                <dbl> <chr>              <dbl>
## 1 A Light in the Attic                  51.8 In stock               3
## 2 Tipping the Velvet                    53.7 In stock               1
## 3 Soumission                            50.1 In stock               1
## 4 Sharp Objects                         47.8 In stock               4
## 5 Sapiens: A Brief History of Humankind 54.2 In stock               5
## 6 The Requiem Red                       22.6 In stock               1
```

Part III: Open Data API

Question 1: Requesting Data

```
# Loading packages for API requests and JSON parsing just like in the lectures

library(httr)
```

```
## Warning: package 'httr' was built under R version 4.5.3
```

```
library(jsonlite)
library(dplyr)
```

```
# API endpoint for Chicago 311 service requests. endpoint is the website

endpoint <- "https://data.cityofchicago.org/resource/v6vf-nfxy.json"

# Request the 1,000 most recent West Town requests from 2026 onward

response <- GET(
  endpoint,
  query = list(
    community_area = "24",
    `$where` = "created_date >= '2026-01-01T00:00:00'",
    `$order` = "created_date DESC",
    `$limit` = 1000
  )
)


# Check the response status. Let's hope to get a 200 code. yes !!
status_code(response)
```

```
## [1] 200
```

##Question 2: Parse response-Convert the JSON response into a data frame

```
# Convert JSON response into the actual data frame

requests_df <- fromJSON(
  rawToChar(response$content)
)


# Display number of rows

nrow(requests_df)
```

```
## [1] 1000
```

```
# Display column names

names(requests_df)
```

```
##  [1] "sr_number"           "sr_type"             "sr_short_code"
##  [4] "owner_department"    "status"              "origin"
##  [7] "created_date"        "last_modified_date"  "street_address"
## [10] "city"                "state"               "zip_code"
## [13] "street_number"       "street_direction"    "street_name"
## [16] "street_type"         "duplicate"           "legacy_record"
## [19] "community_area"      "ward"                "electrical_district"
## [22] "electricity_grid"    "police_sector"       "police_district"
## [25] "police_beat"         "precinct"            "created_hour"
## [28] "created_day_of_week" "created_month"       "x_coordinate"
## [31] "y_coordinate"        "latitude"            "longitude"
## [34] "location"            "created_department"  "closed_date"
## [37] "parent_sr_number"
```

QUestion 3: Summarize Response

```
# Count service requests by type and display the 10 most common ones

top_requests <- requests_df %>%
  count(sr_type, sort = TRUE) %>%
  head(10)

top_requests
```

```
##                                    sr_type   n
## 1                Graffiti Removal Request 203
## 2             Rodent Baiting/Rat Complaint  52
## 3  Finance Parking Code Enforcement Review  47
## 4               Yard Waste Pick-Up Request  43
## 5              Pothole in Street Complaint  37
## 6    Sign Repair Request - All Other Signs  34
## 7               Street Light Out Complaint  31
## 8                           Tree Emergency  31
## 9              Abandoned Vehicle Complaint  28
## 10               Garbage Cart Maintenance  27
```

Question 4: Interpret the results

The 311 service request show that the top request in 2026 was Graffiti removal request with 218. Rodent bait came second with 53 and Finance Parking Code Enforcement review third with 45. The remaining seven, were relatively closer in request count.

I think what the data doesn't show could be problems that people do not report or that are not in the top 1000.

BONUS PART: Median completion time

```
# Identify the 5 most common request types
top5_types <- requests_df %>%
  count(sr_type, sort = TRUE) %>%
  slice(1:5)

top5_types
```

```
##                                   sr_type   n
## 1               Graffiti Removal Request 203
## 2            Rodent Baiting/Rat Complaint  52
## 3 Finance Parking Code Enforcement Review  47
## 4              Yard Waste Pick-Up Request  43
## 5             Pothole in Street Complaint  37
```

```
#Store the names of the top 5 requests

top5_names <- top5_types$sr_type

top5_names
```

```
## [1] "Graffiti Removal Request"
## [2] "Rodent Baiting/Rat Complaint"
## [3] "Finance Parking Code Enforcement Review"
## [4] "Yard Waste Pick-Up Request"
## [5] "Pothole in Street Complaint"
```

```
# Now attempting to build the table

bonus_tbl <- requests_df %>%
  filter(status == "Completed") %>%
  mutate(
    created_date = as.Date(created_date),
    closed_date = as.Date(closed_date),
    days_to_close = as.numeric(closed_date - created_date)
  ) %>%
  filter(sr_type %in% top5_names) %>%
  group_by(sr_type) %>%
  summarise(
    median_days = median(days_to_close, na.rm = TRUE)
  ) %>%
  arrange(desc(median_days))

bonus_tbl
```

```
## # A tibble: 5 × 2
##   sr_type                                median_days
##   <chr>                                        <dbl>
## 1 Pothole in Street Complaint                      5
## 2 Graffiti Removal Request                         1
## 3 Rodent Baiting/Rat Complaint                     1
## 4 Yard Waste Pick-Up Request                       1
## 5 Finance Parking Code Enforcement Review          0
```

```
#double checking the request statuses

unique(requests_df$status)
```

```
## [1] "Open"      "Completed" "Canceled"
```

Part IV: Census API

Step 1: Identify the Variables

```
library(tidycensus)
```

```
## Warning: package 'tidycensus' was built under R version 4.5.3
```

```
# Loading the ACS variable metadata
acs_vars <- load_variables(
  2024,
  "acs5",
  cache = TRUE
)
```

```
## Warning: `cache` is deprecated and ignored. tidycensus no longer writes
## variable metadata to a local cache.
```

```
head(acs_vars)
```

```
## # A tibble: 6 × 4
##   name        label                                    concept          geography
##   <chr>       <chr>                                    <chr>            <chr>
## 1 B01001A_001 Estimate!!Total:                         Sex by Age (Whi… tract
## 2 B01001A_002 Estimate!!Total:!!Male:                  Sex by Age (Whi… tract
## 3 B01001A_003 Estimate!!Total:!!Male:!!Under 5 years  Sex by Age (Whi… tract
## 4 B01001A_004 Estimate!!Total:!!Male:!!5 to 9 years   Sex by Age (Whi… tract
## 5 B01001A_005 Estimate!!Total:!!Male:!!10 to 14 years Sex by Age (Whi… tract
## 6 B01001A_006 Estimate!!Total:!!Male:!!15 to 17 years Sex by Age (Whi… tract
```

```
acs_vars %>%
  filter(grepl("50.0 percent or more", label))
```

```
## # A tibble: 18 × 4
##    name       label                                                concept geography
##    <chr>      <chr>                                                <chr>   <chr>
##  1 B25070_010 Estimate!!Total:!!50.0 percent or more               Gross … block gr…
##  2 B25074_009 Estimate!!Total:!!Less than $10,000:!!50.0 perc…     Househ… block gr…
##  3 B25074_018 Estimate!!Total:!!$10,000 to $19,999:!!50.0 per…    Househ… block gr…
##  4 B25074_027 Estimate!!Total:!!$20,000 to $34,999:!!50.0 per…    Househ… block gr…
##  5 B25074_036 Estimate!!Total:!!$35,000 to $49,999:!!50.0 per…    Househ… block gr…
##  6 B25074_045 Estimate!!Total:!!$50,000 to $74,999:!!50.0 per…    Househ… block gr…
##  7 B25074_054 Estimate!!Total:!!$75,000 to $99,999:!!50.0 per…    Househ… block gr…
##  8 B25074_063 Estimate!!Total:!!$100,000 or more:!!50.0 perce…    Househ… block gr…
##  9 B25091_011 Estimate!!Total:!!Housing units with a mortgage…    Mortga… block gr…
## 10 B25091_022 Estimate!!Total:!!Housing units without a mortg…    Mortga… block gr…
## 11 B25095_009 Estimate!!Total:!!Less than $10,000:!!50.0 perc…    Househ… tract
## 12 B25095_018 Estimate!!Total:!!$10,000 to $19,999:!!50.0 per…    Househ… tract
## 13 B25095_027 Estimate!!Total:!!$20,000 to $34,999:!!50.0 per…    Househ… tract
## 14 B25095_036 Estimate!!Total:!!$35,000 to $49,999:!!50.0 per…    Househ… tract
## 15 B25095_045 Estimate!!Total:!!$50,000 to $74,999:!!50.0 per…    Househ… tract
## 16 B25095_054 Estimate!!Total:!!$75,000 to $99,999:!!50.0 per…    Househ… tract
## 17 B25095_063 Estimate!!Total:!!$100,000 to $149,999:!!50.0 p…    Househ… tract
## 18 B25095_072 Estimate!!Total:!!$150,000 or more:!!50.0 perce…    Househ… tract
```

```
# Let's find the Median Household Income. Which one is it though?

acs_vars %>%
  filter(grepl("Median household income", label))
```

```
## # A tibble: 25 × 4
##    name        label                                               concept geography
##    <chr>       <chr>                                               <chr>   <chr>
##  1 B19013A_001 Estimate!!Median household income in the past …     Median… tract
##  2 B19013B_001 Estimate!!Median household income in the past …     Median… tract
##  3 B19013C_001 Estimate!!Median household income in the past …     Median… tract
##  4 B19013D_001 Estimate!!Median household income in the past …     Median… tract
##  5 B19013E_001 Estimate!!Median household income in the past …     Median… county
##  6 B19013F_001 Estimate!!Median household income in the past …     Median… tract
##  7 B19013G_001 Estimate!!Median household income in the past …     Median… tract
##  8 B19013H_001 Estimate!!Median household income in the past …     Median… tract
##  9 B19013I_001 Estimate!!Median household income in the past …     Median… tract
## 10 B19013_001  Estimate!!Median household income in the past …     Median… block gr…
## # ℹ 15 more rows
```

```
# Let's find the Median Household Income
acs_vars %>%
  filter(name == "B19013_001")
```

```
## # A tibble: 1 × 4
##   name       label                                                concept geography
##   <chr>      <chr>                                                <chr>   <chr>
## 1 B19013_001 Estimate!!Median household income in the past 12… Median… block gr…
```

Gross Rent as a Percentage of Household Income Total: B25070_001E

Gross Rent as a Percentage of Household Income 50.0 percent or more: B25070_010E

Median Household Income: B19013_001E

Step 2: Retrieve the data

```
# Retrieve ACS data for all Ohio counties as requested
ohio_data <- get_acs(
  geography = "county",
  state = "OH",
  variables = c(
    total_renters = "B25070_001",
    severe_rent_burden = "B25070_010",
    median_income = "B19013_001"
  ),
  year = 2024
)
```

```
## Getting data from the 2020-2024 5-year ACS
```

```
# Display first rows
head(ohio_data)
```

```
## # A tibble: 6 × 5
##   GEOID NAME               variable           estimate   moe
##   <chr> <chr>              <chr>                 <dbl> <dbl>
## 1 39001 Adams County, Ohio median_income         50264  4725
## 2 39001 Adams County, Ohio total_renters          2954   317
## 3 39001 Adams County, Ohio severe_rent_burden      559   167
## 4 39003 Allen County, Ohio median_income         64038  1778
## 5 39003 Allen County, Ohio total_renters         13343   692
## 6 39003 Allen County, Ohio severe_rent_burden     2537   354
```

Step 3: Clean and Transform

Step 3.1: Create a wide format of the data

```
# Convert from long format to wide format.

library(tidyverse)
```

```
## Warning: package 'tidyverse' was built under R version 4.5.2
```

```
## Warning: package 'ggplot2' was built under R version 4.5.3
```

```
## Warning: package 'tibble' was built under R version 4.5.3
```

```
## Warning: package 'tidyr' was built under R version 4.5.3
```

```
## Warning: package 'readr' was built under R version 4.5.3
```

```
## Warning: package 'purrr' was built under R version 4.5.3
```

```
## Warning: package 'forcats' was built under R version 4.5.3
```

```
## Warning: package 'lubridate' was built under R version 4.5.3
```

```
## ── Attaching core tidyverse packages ──────────────────────── tidyverse 2.0.0 ──
## ✓ forcats   1.0.1     ✓ readr     2.2.0
## ✓ ggplot2   4.0.3     ✓ tibble    3.3.1
## ✓ lubridate 1.9.5     ✓ tidyr     1.3.2
## ✓ purrr     1.2.2
## ── Conflicts ────────────────────────────────────────── tidyverse_conflicts() ──
## ✖ dplyr::filter()         masks stats::filter()
## ✖ purrr::flatten()        masks jsonlite::flatten()
## ✖ readr::guess_encoding() masks rvest::guess_encoding()
## ✖ dplyr::lag()            masks stats::lag()
## ℹ Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflict
s to become errors
```

```
ohio_wide <- ohio_data %>%
  select(NAME, variable, estimate) %>%
  pivot_wider(
    names_from = variable,
    values_from = estimate
  )

# Display first rows
head(ohio_wide)
```

```
## # A tibble: 6 × 4
##   NAME                   median_income total_renters severe_rent_burden
##   <chr>                          <dbl>         <dbl>               <dbl>
## 1 Adams County, Ohio             50264          2954                 559
## 2 Allen County, Ohio             64038         13343                2537
## 3 Ashland County, Ohio           69860          4589                 997
## 4 Ashtabula County, Ohio         57728          9556                2046
## 5 Athens County, Ohio            56001          9115                2751
## 6 Auglaize County, Ohio          78660          4146                 582
```

Step 3.2: Create Severe Burden Rate

```
# Calculate severe rent burden rate at 50% or more

ohio_wide <- ohio_wide %>%
  mutate(
    severe_burden_rate =
      (severe_rent_burden / total_renters) * 100
  )

# Checking  first rows
head(ohio_wide)
```

```
## # A tibble: 6 × 5
##   NAME         median_income total_renters severe_rent_burden severe_burden_rate
##   <chr>                <dbl>         <dbl>              <dbl>               <dbl>
## 1 Adams Count…         50264          2954                559                18.9
## 2 Allen Count…         64038         13343               2537                19.0
## 3 Ashland Cou…         69860          4589                997                21.7
## 4 Ashtabula C…         57728          9556               2046                21.4
## 5 Athens Coun…         56001          9115               2751                30.2
## 6 Auglaize Co…         78660          4146                582                14.0
```

Step 3.3: Arrange counties form highest to lowerst rate

```
# Sort counties by severe rent burden rate
ohio_wide <- ohio_wide %>%
  arrange(desc(severe_burden_rate))

# Display first rows
head(ohio_wide)
```

```
## # A tibble: 6 × 5
##   NAME         median_income total_renters severe_rent_burden severe_burden_rate
##   <chr>                <dbl>         <dbl>              <dbl>               <dbl>
## 1 Athens Coun…         56001          9115               2751                30.2
## 2 Portage Cou…         75766         19481               5410                27.8
## 3 Hamilton Co…         72470        144332              35228                24.4
## 4 Scioto Coun…         50609          9115               2194                24.1
## 5 Lorain Coun…         73347         33139               7971                24.1
## 6 Butler Coun…         81590         44147              10603                24.0
```

BONUS: Split the NAME column into COUNTY and STATE columns

```
# Split NAME into COUNTY and STATE columns. Basically add more columns.
ohio_wide <- ohio_wide %>%
  separate(
    NAME,
    into = c("county", "state"),
    sep = ", "
  )

# Display first rows
head(ohio_wide)
```

```
## # A tibble: 6 × 6
##   county state median_income total_renters severe_rent_burden severe_burden_rate
##   <chr>  <chr>         <dbl>         <dbl>               <dbl>               <dbl>
## 1 Athen… Ohio          56001          9115                2751                30.2
## 2 Porta… Ohio          75766         19481                5410                27.8
## 3 Hamil… Ohio          72470        144332               35228                24.4
## 4 Sciot… Ohio          50609          9115                2194                24.1
## 5 Lorai… Ohio          73347         33139                7971                24.1
## 6 Butle… Ohio          81590         44147               10603                24.0
```

Step 4: Analyze

Step 4.1: Mean and Median of severe_burden_rate across counties

```
# computing the mean and median

mean(ohio_wide$severe_burden_rate)
```

```
## [1] 18.28335
```

```
median(ohio_wide$severe_burden_rate)
```

```
## [1] 18.66656
```

```
# Top 5 counties with highest severe rent burden rates

top5_counties <- ohio_wide %>%
  arrange(desc(severe_burden_rate)) %>%
  slice(1:5)

top5_counties
```

```
## # A tibble: 5 × 6
##   county state median_income total_renters severe_rent_burden severe_burden_rate
##   <chr>  <chr>         <dbl>         <dbl>              <dbl>               <dbl>
## 1 Athen… Ohio          56001          9115               2751                30.2
## 2 Porta… Ohio          75766         19481               5410                27.8
## 3 Hamil… Ohio          72470        144332              35228                24.4
## 4 Sciot… Ohio          50609          9115               2194                24.1
## 5 Lorai… Ohio          73347         33139               7971                24.1
```

Step 4.2: Identify the 5 counties with the highest rates and the 5 with the lowest rates.

```
# Bottom 5 counties with lowest severe rent burden rates

bottom5_counties <- ohio_wide %>%
  arrange(severe_burden_rate) %>%
  slice(1:5)

bottom5_counties
```

```
## # A tibble: 5 × 6
##   county state median_income total_renters severe_rent_burden severe_burden_rate
##   <chr>  <chr>         <dbl>         <dbl>              <dbl>               <dbl>
## 1 Noble… Ohio          56098           760                 23                3.03
## 2 Merce… Ohio          76782          3355                261                7.78
## 3 Champ… Ohio          75556          3629                313                8.62
## 4 Wyand… Ohio          69768          2327                247               10.6
## 5 Union… Ohio         112322          5046                589               11.7
```

Step 5: Visualize

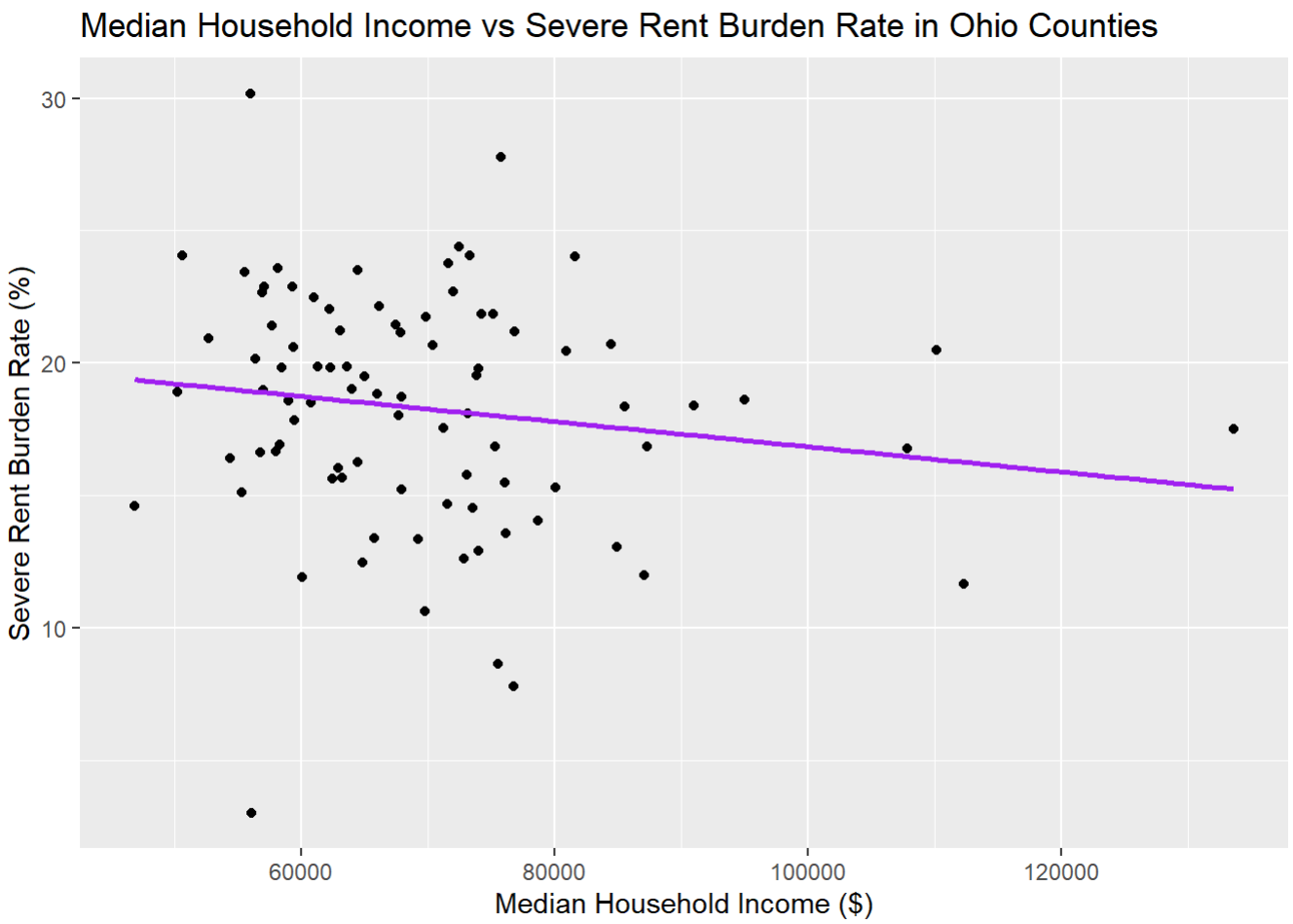
Step 5.1 : Scatterplot of Median Household Income and Severe Rent Burden

```
#loading required library for visualization

library(ggplot2)

ggplot(
  ohio_wide,
  aes(
    x = median_income,
    y = severe_burden_rate
  )
) +
  geom_point() +
  geom_smooth(method = "lm", se = FALSE, color = "purple") +
  labs(
    title = "Median Household Income vs Severe Rent Burden Rate in Ohio Counties",
    x = "Median Household Income ($)",
    y = "Severe Rent Burden Rate (%)"
  )
```

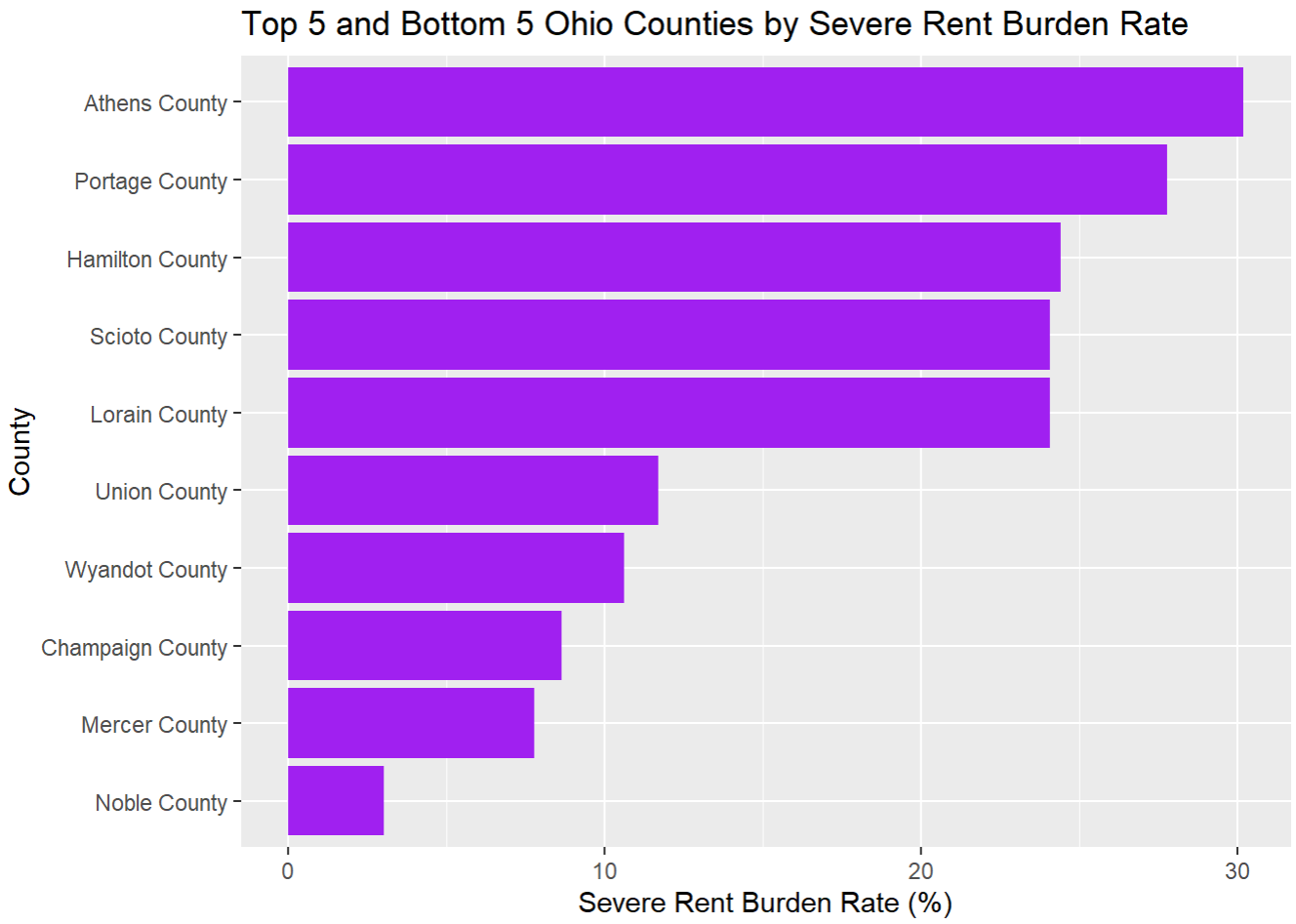
```
## `geom_smooth()` using formula = 'y ~ x'
```



Step 5.2: Bar chart of the top 5 and bottom 5 counties from Step 4, with bars in order

```
# Combine top 5 and bottom 5 counties
county_chart <- bind_rows(
  top5_counties,
  bottom5_counties
)

# Create the bar chart
ggplot(
  county_chart,
  aes(
    x = reorder(county, severe_burden_rate),
    y = severe_burden_rate
  )
) +
  geom_col(fill = "purple") +
  coord_flip() +
  labs(
    title = "Top 5 and Bottom 5 Ohio Counties by Severe Rent Burden Rate",
    x = "County",
    y = "Severe Rent Burden Rate (%)"
  )
```



Step 6: Reflection

Step 6.1 : What relationship do you see between income and severe rent burden in Ohio, and what might explain it?

The linear models suggests that as income increases the severe burden rate slighly decreased, meaning that those counties are spending not spending 50% or more of their income in rent. Not as much as I would think but the relationpship appears to be slightly negative.

One possible explanation can simply be income.Higher income households have more financial resources available to cover housing cost.

Step 6.2 : What are two limitations of using county-level ACS estimates to decide where to target rental assistance?

Hmm.. I think that counties vary by size and conditions might vary within its' own county. For example, I visited the City of Aurora (Kane County) and the contrast is extreme when it comes to the affluent vs. the blue collar areas of Aurora.

Having said that, I tied that if county average data can hide the true needs of all their citizens when it comes to rental assistance. Policy makers should not only use average county data but use supplemental data to make decisions.

Another reason is because they are survey estimates with margins of error, not complete counts of every household.

Part V: Civic Data Ethics

Question 1: Privacy

Since the data includes physical addressed and coordinates, this is delicate as it can identify and provide more information about a specific household or coordinate. E.g. number of complaints submitted by address 1234 W Home St. etc. To deidentify, I would remove the address and possibly aggregate the complaints by neighborhood (e.g. Wicker Park, Bucktown, Chatham etc.)

QUestion 2: Bias and Data Quality

I think that a city manager that assigns repairs solely on request counts could create an uneven service allocation to a city and not by intent. If more report issues in Wicker Park than in Lawndale, then those repairs will mostly to go Wicker Part (which is realistically possible). This means the same issues might occur in Lawndale but given the demographics, less report might be generated. To mitigate for this I would say that a city manager should look at maybe infrastructure maintenance logs to see if those match the request counts. If they don't, further investigation might be warranted before dispatching crews.

Question 3: Scrapping Responsibly

I think the web scraping activities performed in Part II were appropriate because the information was publicly available, factual and did not contain personally identifiable information as you described in our lectures. The Wikipedia page the Books to Scrape page did not contain private info.

One civic data situation when scrapping responsibly is when we are accessing personable identifiable information and a log in with password is necessary. I think a good example would be LINKEDIN, as this page contains name, email, employment and education information that can be used to further get more detailed information.

AI Use Statement

I used Copilot to assist me with code revision and debugging as I progressed through the exam This helped me think of alternatives when writing the code for a better output. The assistance included help with web

scrapping and the API part (as this was challenging honestly).I find this material very fascinating, though challenging. All work was executed, reviewed and honestly submitted with my efforts.